

PROMPT TÉCNICO PARA ANTIGRAVITY - MÓDULO 2: SISTEMA DE RESERVAS (FRONTEND DEL CLIENTE)

markdown

CONTEXTO DEL MÓDULO 2

Estás construyendo el FRONTEND PÚBLICO de reservas para restaurantes (la página que ven los clientes finales). Este módulo se integra con Cal.com como backend de reservas (self-hosted).

El flujo debe ser: Selección de Sede → Número de Personas → Fecha/Hora → Datos → Pago (si aplica) → Confirmación.

STACK TECNOLÓGICO:

- Frontend: Next.js 14+ (App Router), TypeScript, TailwindCSS
- UI Components: shadcn/ui, Radix UI
- Calendar: react-day-picker + date-fns
- Maps: @react-google-maps/api o Leaflet
- Forms: React Hook Form + Zod validation
- State Management: Zustand (para el flujo multi-paso)
- Animations: framer-motion (transiciones suaves entre pasos)
- QR Code: qrcode.react
- Payments: SDKs de Wompi/Stripe/PayU según configuración
- API Client: Axios o fetch nativo con SWR/React Query
- Internationalization: next-intl (multi-idiotoma)
- Real-time: WebSocket o Server-Sent Events para disponibilidad

ARQUITECTURA DEL FLUJO DE RESERVA

ESTRUCTURA DE PASOS:

Paso 0: Selección de Restaurante/Sede (si es cadena)

↓

Paso 1: Número de Personas + Tipo de Evento

↓

Paso 2: Fecha y Hora Disponible

↓

Paso 3: Datos del Cliente (Guest o Login)

↓

Paso 4: Confirmación y Pago (si aplica depósito)



Paso 5: Éxito + QR de Confirmación

****ESTADO GLOBAL (Zustand Store):****

```
``typescript
interface BookingState {
  // Paso 0
  restaurant: Restaurant | null;
  branch: Branch | null;

  // Paso 1
  partySize: number;
  eventType: string;
  specialRequests: string;

  // Paso 2
  selectedDate: Date | null;
  selectedTime: string | null;

  // Paso 3
  customer: {
    name: string;
    email: string;
    phone: string;
    isNewCustomer: boolean;
  };

  // Paso 4
  paymentMethod: string | null;
  depositAmount: number | null;

  // Paso 5
  bookingId: string | null;
  qrCode: string | null;

  // Metadata
  currentStep: number;
  isProcessing: boolean;
  errors: Record<string, string>;

  // Actions
  setRestaurant: (restaurant: Restaurant) => void;
```

```

setBranch: (branch: Branch) => void;
setPartySize: (size: number) => void;
setEventType: (type: string) => void;
setSpecialRequests: (requests: string) => void;
setSelectedDate: (date: Date) => void;
setSelectedTime: (time: string) => void;
setCustomer: (customer: Partial<Customer>) => void;
setPaymentMethod: (method: string) => void;
setBookingId: (id: string) => void;
setCurrentStep: (step: number) => void;
reset: () => void;
}

```

PASO 0: SELECCIÓN DE RESTAURANTE/SEDE

REQUISITOS FUNCIONALES:

Pantalla inicial que muestra las opciones disponibles para el cliente.

ESCENARIO A: Un solo restaurante (sin sucursales)

- Redirige automáticamente al Paso 1
- No muestra esta pantalla

ESCENARIO B: Cadena con múltiples sedes

- Grid de cards visuales (2 columnas en móvil, 3-4 en desktop)
- Cada card muestra:
 - Foto principal de la sede (16:9 aspect ratio)
 - Nombre de la sede
 - Dirección completa
 - Distancia desde la ubicación del usuario (si permite geolocalización)
 - Rating promedio (estrellas)
 - Horario actual (Abierto/Cerrado + hora de cierre)
 - Badge "Más popular" o "Cerca de ti" si aplica
- Barra de búsqueda por ciudad/barrio
- Filtros:
 - "Cerca de mí" (geolocalización)
 - "Abierto ahora"
 - "Con disponibilidad hoy"
- Ordenamiento:
 - Más cercano (default si geolocalización)
 - Más popular
 - Mejor rating
- Click en card → selecciona sede y avanza al Paso 1
- Botón "Ver todas las sedes" si hay más de 6

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// BranchSelector.tsx
interface BranchSelectorProps {
  branches: Branch[];
  onSelect: (branch: Branch) => void;
  userLocation?: { lat: number; lng: number };
}

// BranchCard.tsx
interface BranchCardProps {
  branch: Branch;
  distance?: number;
  isSelected?: boolean;
  onClick: () => void;
}

// BranchFilters.tsx
interface BranchFiltersProps {
  onFilterChange: (filters: BranchFilters) => void;
  filters: BranchFilters;
}
```

API Endpoints:

```
typescript
// GET /api/branches?restaurantId={id}
Response: {
  branches: Array<{
    id: string,
    name: string,
    address: string,
    city: string,
    lat: number,
    lng: number,
    photoUrl: string,
    rating: number,
    reviewCount: number,
    openingHours: {
      dayOfWeek: number,
      shifts: Array<{open: string, close: string}>,
    }[],
    isOpenNow: boolean,
    hasAvailabilityToday: boolean,
  }>
}
```

```
}
```

```
// GET /api/branches/{id}/availability?date={YYYY-MM-DD}
```

```
Response: {
```

```
  availableSlots: string[], // ["12:00", "12:30", "13:00",  
  ...]
```

```
  totalCapacity: number,
```

```
  bookedCount: number,
```

```
}
```

Cálculo de Distancia:

typescript

```
import { getDistance } from 'geolib';
```

```
function calculateDistance(  
  userLocation: { lat: number; lng: number },  
  branchLocation: { lat: number; lng: number }  
): number {
```

```
  return getDistance(userLocation, branchLocation); // metros
```

```
}
```

```
function formatDistance(meters: number): string {  
  if (meters < 1000) return `${Math.round(meters)}m`;  
  return `${(meters / 1000).toFixed(1)}km`;  
}
```

```
}
```

```
function formatDistance(meters: number): string {
```

```
  if (meters < 1000) return `${Math.round(meters)}m`;
```

```
  return `${(meters / 1000).toFixed(1)}km`;
```

```
}
```

UI/UX:

- Skeleton loaders mientras cargan las sedes
- Animación de fade-in al aparecer las cards
- Hover effect en desktop (scale 1.02 + shadow)
- Click feedback (ripple effect)
- Responsive: 1 columna móvil, 2 tablet, 3-4 desktop
- Accesibilidad: ARIA labels, keyboard navigation (Tab + Enter)

PASO 1: NÚMERO DE PERSONAS + TIPO DE EVENTO

REQUISITOS FUNCIONALES:

Pantalla para definir el tamaño del grupo y el tipo de ocasión.

SECCIÓN 1: NÚMERO DE PERSONAS

- Pregunta: "¿Para cuántas personas?"
- Selector visual (NO dropdown):
 - Botones grandes y táctiles: 1, 2, 3, 4, 5, 6, 7, 8

- Botón "+ Más personas" (abre input numérico)
- Input numérico con min: 1, max: capacidad máxima del restaurante
- Validación: no puede ser mayor a la capacidad máxima configurada
- Indicador visual: icono de personas que cambia según la selección
- Mensaje dinámico:
 - Si selecciona 1-2: "Mesa romántica o individual"
 - Si selecciona 3-6: "Mesa para grupo"
 - Si selecciona 7-10: "Mesa grande"
 - Si selecciona +10: "Evento especial (requiere confirmación)"

SECCIÓN 2: TIPO DE EVENTO (OPCIONAL)

- Pregunta: "¿Qué tipo de ocasión es?"
- Selector de opciones (cards con iconos):
 - 🍴 Cena casual (default)
 - 💕 Cena romántica
 - 🎂 Cumpleaños
 - 💼 Reunión de negocios
 - 🎉 Celebración especial
 - 👨‍👩‍👧 Comida familiar
 - 🥂 Aniversario
 - Otro (input libre)
- Selección múltiple NO permitida
- Campo opcional (puede saltarlo)

SECCIÓN 3: SOLICITUDES ESPECIALES (OPCIONAL)

- Textarea: "¿Alguna solicitud especial?"
- Placeholder: "Ej: Silla alta para bebé, alergias, mesa cerca de la ventana..."
- Max: 500 caracteres
- Contador de caracteres
- Chips de sugerencias rápidas (click para agregar):
 - "Silla alta"
 - "Accesibilidad wheelchair"
 - "Mesa tranquila"
 - "Vista panorámica"
 - "Alergias alimentarias" (abre input para especificar)

VALIDACIONES:

- Número de personas: requerido, min 1, max capacidad del restaurante
- Si supera capacidad máxima: mostrar mensaje "Para grupos de más de X personas, por favor contáctanos directamente"
- Tipo de evento: opcional
- Solicitudes especiales: opcional, max 500 chars

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// PartySizeSelector.tsx
interface PartySizeSelectorProps {
  value: number;
  onChange: (size: number) => void;
  maxCapacity: number;
  disabled?: boolean;
}

// EventTypeSelector.tsx
interface EventTypeSelectorProps {
  value: string;
  onChange: (type: string) => void;
}

// SpecialRequests.tsx
interface SpecialRequestsProps {
  value: string;
  onChange: (requests: string) => void;
  maxLength?: number;
}

// StepOne.tsx (contenedor principal)
```

Validaciones con Zod:

```
typescript
import { z } from 'zod';

export const stepOneSchema = z.object({
  partySize: z.number()
    .min(1, "Debe seleccionar al menos 1 persona")
    .max(50, "El grupo es demasiado grande"),
  eventType: z.string().optional(),
  specialRequests: z.string().max(500, "Máximo 500 caracteres").optional(),
});

export type StepOneData = z.infer<typeof stepOneSchema>;
UI/UX:
```

- Botones de número de personas: 48x48px mínimo (táctil)

- Animación de selección (bounce effect)
- Transición suave al cambiar de paso (framer-motion)
- Mensajes de error inline (debajo del campo)
- Tooltip explicativo en "Más personas"
- Iconos SVG inline (no dependencias externas pesadas)
- Accesibilidad: ARIA labels, keyboard navigation, focus visible

PASO 2: FECHA Y HORA DISPONIBLE

REQUISITOS FUNCIONALES:

Pantalla para seleccionar fecha y hora de la reserva.

VISTA UNIFICADA (NO dos pasos separados):

En lugar de tener "Paso 2A: Fecha" y "Paso 2B: Hora", mostramos AMBOS en la misma pantalla.

LAYOUT (Desktop):

Calendario (Izquierda)	Horarios (Derecha)
[Lun Mar Mie Jue Vie]	[12:00]
[1 2 3 4 5]	[12:30]
[6 7 8 9 10]	[13:00]
[...]	[13:30]
	[...]

LAYOUT (Móvil):

Calendario	
[Lun Mar Mie Jue]	
[1 2 3 4]	
[5 6 7 8]	
Horarios	
[12:00] [12:30]	
[13:00] [13:30]	
[14:00] [14:30]	

SECCIÓN 1: CALENDARIO

- Componente: react-day-picker
- Rango de fechas:
 - Mínimo: Hoy (o mañana si es muy tarde para reservar hoy)
 - Máximo: 60 días en el futuro (configurable por restaurante)
- Indicadores visuales en cada día:
 -  Verde: Disponible (hay horarios libres)
 -  Amarillo: Poca disponibilidad (últimas mesas)
 -  Rojo: Lleno (no hay disponibilidad)
 -  Gris: Cerrado (el restaurante no abre ese día)
- Navegación:
 - Flechas para cambiar de mes
 - Click en día → carga los horarios disponibles en el panel derecho
 - Días pasados: deshabilitados
- Selector de "Vista rápida":
 - Botones: "Hoy", "Mañana", "Próximos 7 días"
 - Click en "Hoy" → selecciona hoy y carga horarios

SECCIÓN 2: HORARIOS DISPONIBLES

- Grid de time slots (botones)
- Cada slot muestra:
 - Hora (formato 12h o 24h según configuración)
 - Badge si es "Última mesa" o "Happy Hour"
- Estados de cada slot:
 - Disponible: botón normal, clickable
 - Última mesa: badge amarillo, clickable
 - Lleno: botón deshabilitado, tachado
 - Seleccionado: botón con color primario, checkmark
- Ordenamiento:
 - Cronológico (12:00, 12:30, 13:00, ...)
 - Agrupación por turnos si aplica:
 - "Almuerzo" (12:00 - 15:00)
 - "Cena" (19:00 - 23:00)
- Filtros (opcional):
 - "Solo almuerzo"
 - "Solo cena"
 - "Horarios con descuento"
- Scroll horizontal si hay muchos horarios

SECCIÓN 3: INFORMACIÓN ADICIONAL

- Mensaje dinámico según selección:
 - "Has seleccionado: Viernes 15 de Marzo a las 20:00 para 4 personas"
- Duración estimada de la reserva (ej: "Duración aproximada: 1h 30m")
- Política de cancelación (ej: "Cancelación gratuita hasta 2h antes")

- Mensaje si no hay disponibilidad:
 - "Lo sentimos, no hay disponibilidad para esa fecha"
 - Sugerencia: "¿Te parece bien el sábado a las 19:30?"
 - Botón "Ver otras fechas"

VALIDACIONES:

- Fecha: requerida, no puede ser pasada
- Hora: requerida, debe estar en la lista de disponibles
- Si cambia la fecha, se resetea la hora seleccionada
- Si cambia el número de personas, se recalcula la disponibilidad

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// DateTimeSelector.tsx (contenedor principal)
interface DateTimeSelectorProps {
  restaurantId: string;
  branchId: string;
  partySize: number;
  selectedDate: Date | null;
  selectedTime: string | null;
  onChangeDate: (date: Date) => void;
  onChangeTime: (time: string) => void;
}

// AvailabilityCalendar.tsx
interface AvailabilityCalendarProps {
  restaurantId: string;
  branchId: string;
  partySize: number;
  selectedDate: Date | null;
  onSelectDate: (date: Date) => void;
  minDate?: Date;
  maxDate?: Date;
}

// TimeSlotsGrid.tsx
interface TimeSlotsGridProps {
  date: Date;
  availableSlots: TimeSlot[];
  selectedTime: string | null;
  onSelectTime: (time: string) => void;
```

```
    timeFormat?: '12h' | '24h';  
  }
```

```
// TimeSlotButton.tsx  
interface TimeSlotButtonProps {  
  time: string;  
  isAvailable: boolean;  
  isSelected: boolean;  
  isLastTable?: boolean;  
  onClick: () => void;  
}
```

API Endpoints:

typescript

```
// GET /api/restaurants/{id}/branches/{branchId}/availability
```

Query params:

- startDate: YYYY-MM-DD
- endDate: YYYY-MM-DD
- partySize: number

```
Response: {  
  dates: {  
    [date: string]: {  
      status: 'available' | 'limited' | 'full' | 'closed',  
      slots: Array<{  
        time: string, // "12:00"  
        available: boolean,  
        isLastTable: boolean,  
        hasDiscount: boolean,  
      }>  
    }  
  }  
}
```

```
// GET /api/restaurants/{id}/branches/{branchId}/slots
```

Query params:

- date: YYYY-MM-DD
- partySize: number

```
Response: {  
  slots: Array<{  
    time: string,  
    available: boolean,  
    isLastTable: boolean,  
    hasDiscount: boolean,  
  }>
```

```
    discountPercentage?: number,  
  } >  
}
```

Integración con [Cal.com](https://cal.com):

```
// Función para obtener disponibilidad desde Cal.com  
async function getCalComAvailability(  
  eventTypeId: number,  
  startDate: string,  
  endDate: string  
) : Promise<AvailabilityResponse> {  
  const response = await fetch(  
  
    `${CALCOM_API_URL}/api/v2/availability?eventType=${eventTypeId}&startDate=${  
    startDate}&endDate=${endDate}`,  
    {  
      headers: {  
        'Authorization': `Bearer ${CALCOM_API_KEY}`,  
      },  
    }  
  );  
  
  if (!response.ok) throw new Error('Failed to fetch availability');  
  
  return response.json();  
}
```

UI/UX:

- Skeleton loaders para calendario y horarios
- Animación de transición al cambiar de mes
- Hover effect en días y time slots
- Feedback táctil en móvil (vibración ligera si es soportada)
- Scroll suave al seleccionar un día (en móvil, hace scroll a los horarios)
- Mensajes de error claros si no hay disponibilidad
- Accesibilidad: ARIA labels, keyboard navigation, focus management

PASO 3: DATOS DEL CLIENTE

REQUISITOS FUNCIONALES:

Pantalla para capturar la información del cliente.

ENFOQUE: GUEST-FIRST (sin fricción)

El cliente NO necesita crear una cuenta para reservar.

OPCIÓN A: RESERVAR COMO INVITADO (Default)

Campos requeridos:

- Nombre completo (texto)
- Email (email, validación)
- Teléfono (tel, validación de formato)

Campos opcionales:

- "Crear cuenta para próximas reservas" (checkbox)
 - Si marca: pide contraseña y confirma email
 - Si no marca: solo guarda los datos para esta reserva

OPCIÓN B: LOGIN RÁPIDO (si ya tiene cuenta)

- Botones de login social:
 - Google (OAuth)
 - Facebook (OAuth)
 - Apple (OAuth)
- Login con email + contraseña
- "¿Olvidaste tu contraseña?" (link)
- Autocompletado: si ya reservó antes con ese email, precarga los datos

VALIDACIONES:

- Nombre: requerido, min 2 chars, max 100 chars
- Email: requerido, formato válido, verificación de existencia (opcional)
- Teléfono: requerido, formato válido (validación por país)
- Contraseña (si crea cuenta): min 8 chars, 1 mayúscula, 1 número

CAMPOS ADICIONALES (OPCIONAL):

- "¿Es tu primera vez en este restaurante?" (Sí/No)
- "¿Cómo nos encontraste?" (select: Google, Instagram, Recomendación, Otro)
- "¿Quieres recibir promociones por email?" (checkbox, default NO)

PRIVACIDAD:

- Checkbox: "Acepto la política de privacidad" (requerido)
- Checkbox: "Acepto los términos y condiciones" (requerido)
- Link a documentos legales
- Mensaje: "Tus datos están seguros y nunca serán compartidos"

REQUISITOS TÉCNICOS:

Componentes:

```

typescript
// CustomerForm.tsx
interface CustomerFormProps {
  customer: Customer | null;
  onChange: (customer: Partial<Customer>) => void;
  isGuest: boolean;
  onCreateAccount?: () => void;
  onLogin?: () => void;
}

// SocialLoginButtons.tsx
interface SocialLoginButtonsProps {
  onGoogleLogin: () => void;
  onFacebookLogin: () => void;
  onAppleLogin: () => void;
}

// GuestToggle.tsx
interface GuestToggleProps {
  isGuest: boolean;
  onToggle: () => void;
}

```

Validaciones con Zod:

```

import { z } from 'zod';

export const customerSchema = z.object({
  name: z.string()
    .min(2, "El nombre debe tener al menos 2 caracteres")
    .max(100, "El nombre es demasiado largo"),
  email: z.string().email("Email inválido"),
  phone: z.string()
    .regex(/^\\+?[0-9]{10,15}$/, "Teléfono inválido"),
  password: z.string()
    .min(8, "La contraseña debe tener al menos 8 caracteres")
    .regex(/[A-Z]/, "Debe tener al menos 1 mayúscula")
    .regex(/[0-9]/, "Debe tener al menos 1 número")
    .optional(),
  acceptsPrivacy: z.boolean().refine(val => val === true, {
    message: "Debes aceptar la política de privacidad",
  }),
  acceptsTerms: z.boolean().refine(val => val === true, {
    message: "Debes aceptar los términos y condiciones",
  }),
});

```

```
});
```

```
export type CustomerData = z.infer<typeof customerSchema>;
```

```
Autenticación (NextAuth.js):
```

```
// pages/api/auth/[...nextauth].ts
```

```
import NextAuth from 'next-auth';
```

```
import GoogleProvider from 'next-auth/providers/google';
```

```
import FacebookProvider from 'next-auth/providers/facebook';
```

```
import CredentialsProvider from 'next-auth/providers/credentials';
```

```
export default NextAuth({
```

```
  providers: [
```

```
    GoogleProvider({
```

```
      clientId: process.env.GOOGLE_CLIENT_ID!,
```

```
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
```

```
    }),
```

```
    FacebookProvider({
```

```
      clientId: process.env.FACEBOOK_CLIENT_ID!,
```

```
      clientSecret: process.env.FACEBOOK_CLIENT_SECRET!,
```

```
    }),
```

```
    CredentialsProvider({
```

```
      name: 'Credentials',
```

```
      credentials: {
```

```
        email: { label: "Email", type: "email" },
```

```
        password: { label: "Password", type: "password" }
      },
```

```
    },
```

```
    async authorize(credentials) {
```

```
      // Validar credenciales contra base de datos
```

```
      const user = await prisma.user.findUnique({
```

```
        where: { email: credentials.email },
```

```
      });
```

```
      if (user && await bcrypt.compare(credentials.password, user.password)) {
```

```
        return user;
```

```
      }
```

```
      return null;
```

```
    }
```

```
  }),
```

```
],
```

```
  // ... más configuración
```

```
});
```

```
API Endpoints:
```

```
typescript
```

```
// POST /api/customers
```

```
// Crea o actualiza un cliente
Body: {
  name: string,
  email: string,
  phone: string,
  password?: string, // si crea cuenta
  acceptsPrivacy: boolean,
  acceptsTerms: boolean,
}
Response: {
  customerId: string,
  isNew: boolean,
}

// POST /api/customers/verify-email
Body: { email: string }
Response: {
  exists: boolean,
  suggestedAction: 'login' | 'create_account' |
'continue_as_guest',
}
UI/UX:
```

- Autocompletado de navegador (autocomplete="name", autocomplete="email", etc.)
 - Máscara de teléfono según país (ej: +57 300 123 4567)
 - Validación en tiempo real (onBlur)
 - Indicador de fortaleza de contraseña (si crea cuenta)
 - Toggle entre "Invitado" y "Login" sin recargar página
 - Mensajes de error claros y específicos
 - Accesibilidad: ARIA labels, keyboard navigation, focus management
-

PASO 4: CONFIRMACIÓN Y PAGO

REQUISITOS FUNCIONALES:

Pantalla de resumen y procesamiento de pago (si aplica depósito).

SECCIÓN 1: RESUMEN DE LA RESERVA

Card con toda la información:

-  Restaurante: Nombre + Dirección
-  Fecha: Día de semana, fecha completa
-  Hora: Hora seleccionada

-  Personas: Número de comensales
-  Tipo de evento: (si seleccionó)
-  Solicitudes especiales: (si agregó)
-  Cliente: Nombre + Email + Teléfono
-  Duración estimada: (ej: 1h 30m)
-  Política de cancelación: (ej: "Gratis hasta 2h antes")

Botón: "Editar reserva" (vuelve al paso correspondiente)

SECCIÓN 2: PAGO (SI APLICA DEPÓSITO)

Escenario A: NO requiere depósito

- Mensaje: "No se requiere pago anticipado"
- Botón: "Confirmar reserva"
- Avanza directamente al Paso 5

Escenario B: Requiere depósito

Información del pago:

- Monto del depósito: \$X (o X% del estimado)
- Concepto: "Depósito para reserva el [fecha]"
- Política: "100% consumible en tu visita"
- Política de reembolso: "Reembolso completo si cancelas con 24h de anticipación"

Métodos de pago disponibles:

-  Tarjeta de crédito/débito (default)
-  PSE (Colombia)
-  Efectivo en el local (si el restaurante lo permite)
-  Transferencia bancaria (si aplica)

Formulario de tarjeta:

- Número de tarjeta (con formateo automático: 4242 4242 4242 4242)
- Fecha de expiración (MM/YY)
- CVV (3-4 dígitos)
- Nombre del titular (como aparece en la tarjeta)
- Checkbox: "Guardar tarjeta para futuras reservas" (opcional)

Botones:

- "Pagar \$X y confirmar reserva" (botón principal)
- "Cancelar" (vuelve al paso anterior)

SECCIÓN 3: PROCESAMIENTO

Estados durante el procesamiento:

1. Procesando: Spinner + mensaje "Procesando tu pago..."
2. Verificando: Spinner + mensaje "Verificando disponibilidad..."
3. Confirmando: Spinner + mensaje "Confirmando tu reserva..."
4. Éxito: Redirige al Paso 5
5. Error: Mensaje de error + botón "Reintentar"

Manejo de Errores:

- Tarjeta declinada: "Tu tarjeta fue declinada. Por favor intenta con otra."
- Sin disponibilidad: "Lo sentimos, acaban de reservar esa mesa. Por favor selecciona otro horario."
- Error de pasarela: "Error al procesar el pago. Por favor intenta nuevamente."
- Timeout: "La conexión tardó demasiado. Por favor verifica tu conexión e intenta de nuevo."

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// BookingSummary.tsx
interface BookingSummaryProps {
  booking: BookingState;
  onEdit: (step: number) => void;
}

// PaymentForm.tsx
interface PaymentFormProps {
  amount: number;
  currency: string;
  paymentMethods: string[];
  onPaymentSuccess: (paymentId: string) => void;
  onPaymentError: (error: string) => void;
}

// CreditCardInput.tsx
interface CreditCardInputProps {
  cardNumber: string;
  expiryDate: string;
  cvv: string;
  cardHolder: string;
  onChange: (field: string, value: string) => void;
  errors: Record<string, string>;
}
```

```
// ProcessingOverlay.tsx
interface ProcessingOverlayProps {
  status: 'processing' | 'verifying' | 'confirming' |
'success' | 'error';
  message: string;
}
```

Integración con Pasarelas de Pago:

typescript

// Wompi (Colombia)

```
import { Wompi } from '@wompi/client';
```

```
async function processWompiPayment(
  amount: number,
  cardToken: string,
  customerId: string
): Promise<PaymentResult> {
  const response = await fetch('/api/payments/wompi', {
    method: 'POST',
    body: JSON.stringify({
      amount: amount * 100, // Wompi usa centavos
      currency: 'COP',
      cardToken,
      customerId,
    }),
  });
}
```

```
  return response.json();
}
```

// Stripe (Internacional)

```
import { loadStripe } from '@stripe/stripe-js';
```

```
async function processStripePayment(
  amount: number,
  cardElement: any,
  customerId: string
): Promise<PaymentResult> {
  const stripe = await
loadStripe(process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY!);

  const { paymentMethod, error } = await
stripe.createPaymentMethod({
    type: 'card',
    card: cardElement,
```

```
});
```

```
if (error) throw error;
```

```
const response = await fetch('/api/payments/stripe', {  
  method: 'POST',  
  body: JSON.stringify({  
    amount,  
    currency: 'cop',  
    paymentMethodId: paymentMethod.id,  
    customerId,  
  }),  
});
```

```
return response.json();
```

```
}
```

API Endpoints:

typescript

```
// POST /api/payments/create-intent
```

```
Body: {
```

```
  amount: number,
```

```
  currency: string,
```

```
  paymentMethod: string,
```

```
  bookingData: any,
```

```
}
```

```
Response: {
```

```
  clientSecret: string, // para Stripe
```

```
  transactionId: string,
```

```
}
```

```
// POST /api/payments/confirm
```

```
Body: {
```

```
  transactionId: string,
```

```
  paymentMethodId: string,
```

```
}
```

```
Response: {
```

```
  success: boolean,
```

```
  paymentId: string,
```

```
  error?: string,
```

```
}
```

```
// POST /api/bookings
```

```
Body: {
```

```
  restaurantId: string,
```

```
branchId: string,  
partySize: number,  
eventType?: string,  
specialRequests?: string,  
date: string,  
time: string,  
customerId: string,  
paymentId?: string,  
}  
Response: {  
  bookingId: string,  
  confirmationCode: string,  
  qrCode: string,  
}
```

UI/UX:

- Formateo automático de número de tarjeta (espacios cada 4 dígitos)
- Detección automática de tipo de tarjeta (Visa, Mastercard, Amex)
- Validación en tiempo real de campos de tarjeta
- Máscara de CVV (3 o 4 dígitos según tipo de tarjeta)
- Iconos de tarjetas aceptadas
- Spinner de procesamiento con animación suave
- Mensajes de error específicos (no genéricos)
- Botón de pago deshabilitado mientras procesa
- Accesibilidad: ARIA labels, keyboard navigation

PASO 5: ÉXITO Y QR DE CONFIRMACIÓN

REQUISITOS FUNCIONALES:

Pantalla final de confirmación con el QR de la reserva.

SECCIÓN 1: MENSAJE DE ÉXITO

- Icono grande de checkmark verde (animado)
- Mensaje: "¡Reserva confirmada!"
- Subtítulo: "Te esperamos el [fecha] a las [hora]"
- Código de confirmación: (ej: #RES-2026-001234)

SECCIÓN 2: QR DE CONFIRMACIÓN

- QR code grande y escaneable (min 200x200px)
- Texto debajo: "Muestra este QR al llegar"
- Botones de descarga:

- "Guardar QR" (descarga PNG)
- "Enviar por email" (reenvía al email del cliente)
- "Agregar a Apple Wallet" (si es iOS)
- "Agregar a Google Pay" (si es Android)

SECCIÓN 3: DETALLES DE LA RESERVA

Card con resumen:

- 📍 Restaurante: Nombre + Dirección + Link a Google Maps
- 📅 Fecha y hora
- 👤 Número de personas
- 🎉 Tipo de evento (si aplica)
- 👤 Nombre del cliente
- 💳 Pago: "Depósito de \$X pagado" o "Sin depósito"
- 📄 Política de cancelación
- 📞 Teléfono de contacto del restaurante

SECCIÓN 4: ACCIONES ADICIONALES

Botones:

- "Agregar al calendario" (Google, Apple, Outlook)
- "Compartir reserva" (WhatsApp, Email, SMS)
- "Ver mis reservas" (si tiene cuenta)
- "Volver al inicio"

SECCIÓN 5: RECORDATORIOS

Mensaje: "Te enviaremos un recordatorio por email y WhatsApp 24h y 2h antes de tu reserva"

Checkbox: "No quiero recibir recordatorios" (opcional)

SECCIÓN 6: FEEDBACK

Mensaje: "¿Te fue útil el proceso de reserva?"

- ★★★★★ (rating de 1-5)
- Textarea opcional: "¿Algo que podamos mejorar?"
- Botón: "Enviar feedback"

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// BookingConfirmation.tsx
interface BookingConfirmationProps {
  booking: BookingState;
  qrCode: string;
  onDownloadQR: () => void;
```

```

    onSendByEmail: () => void;
    onAddToCalendar: (provider: 'google' | 'apple' | 'outlook')
=> void;
}

```

```

// QRCodeDisplay.tsx
interface QRCodeDisplayProps {
  value: string;
  size?: number;
  logo?: string;
}

```

```

// CalendarButtons.tsx
interface CalendarButtonsProps {
  booking: BookingState;
  onAddToGoogle: () => void;
  onAddToApple: () => void;
  onAddToOutlook: () => void;
}

```

```

// FeedbackForm.tsx
interface FeedbackFormProps {
  onSubmit: (rating: number, comment?: string) => void;
}

```

Generación de QR:

typescript

```
import QRCode from 'qrcode.react';
```

```

function BookingQR({ bookingId, size = 200 }: { bookingId:
string; size?: number }) {
  const qrValue =
`${process.env.NEXT_PUBLIC_APP_URL}/verify/${bookingId}`;

```

```

  return (
    <QRCode
      value={qrValue}
      size={size}
      level="H" // High error correction
      includeMargin={true}
      imageSettings={{
        src: '/logo.png',
        height: 40,
        width: 40,
        excavate: true,

```

```

    }}
  />
  );
}

```

Agregar al Calendario:

typescript

```

function generateGoogleCalendarUrl(booking: BookingState):
string {
  const startDate = new
Date(`${booking.selectedDate}T${booking.selectedTime}`);
  const endDate = new Date(startDate.getTime() +
booking.restaurant.avgReservationTime * 60000);

  const params = new URLSearchParams({
    action: 'TEMPLATE',
    text: `Reserva en ${booking.restaurant.name}`,
    dates:
`${formatDateForGoogle(startDate)}/${formatDateForGoogle(endDa
te)}`,
    details: `Reserva para ${booking.partySize}
personas\nCódigo: ${booking.bookingId}`,
    location: booking.branch.address,
  });

  return
`https://calendar.google.com/calendar/render?${params.toString
()}`;
}

```

```

function generateICSFile(booking: BookingState): string {
  // Generar archivo .ics para Apple/Outlook
  const ics = `BEGIN:VCALENDAR
VERSION:2.0
BEGIN:VEVENT
DTSTART:${formatDateForICS(startDate)}
DTEND:${formatDateForICS(endDate)}
SUMMARY:Reserva en ${booking.restaurant.name}
DESCRIPTION:Reserva para ${booking.partySize} personas
LOCATION:${booking.branch.address}
END:VEVENT
END:VCALENDAR`;

  return ics;
}

```


API Endpoints:

typescript

```
// POST /api/bookings/{id}/send-email
```

```
Body: { email: string }
```

```
Response: { success: boolean }
```

```
// POST /api/bookings/{id}/feedback
```

```
Body: { rating: number, comment?: string }
```

```
Response: { success: boolean }
```

```
// GET /api/bookings/{id}/qr
```

```
Response: { qrCode: string } // base64
```

UI/UX:

- Animación de confetti al mostrar la confirmación (opcional, sutil)
- QR con animación de "draw" (se dibuja progresivamente)
- Botones de descarga con iconos claros
- Mensaje de éxito con tono celebratorio
- Feedback form con estrellas interactivas
- Accesibilidad: ARIA labels, keyboard navigation

INTEGRACIÓN COMPLETA CON CAL.COM

CONFIGURACIÓN DE CAL.COM:

Event Types necesarios:

typescript

```
// Crear un Event Type por cada tipo de reserva
```

```
const eventTypes = [
```

```
{
```

```
  title: "Reserva Estándar",
```

```
  slug: "reserva-estandar",
```

```
  length: 90, // minutos
```

```
  locations: [{ type: "integrations:zoom" }], // o
```

```
"inPerson"
```

```
  schedulingType: "ROUND_ROBIN",
```

```
  hosts: [], // se asignan dinámicamente
```

```
},
```

```
{
```

```
  title: "Reserva con Depósito",
```

```
  slug: "reserva-con-deposito",
```

```
  length: 90,
```

```

    requiresConfirmation: true,
    metadata: {
      requireDeposit: true,
      depositPercentage: 50,
    },
  },
  // ... más tipos
];

```

Flujo de Creación de Reserva en [Cal.com](#):

typescript

```

async function createCalComBooking(bookingData: BookingState):
Promise<string> {
  const response = await
fetch(`${CALCOM_API_URL}/api/v2/bookings`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${CALCOM_API_KEY}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    eventType: getId(bookingData),
    start:
`${bookingData.selectedDate}T${bookingData.selectedTime}:00Z`,
    end: calculateEndTime(bookingData),
    responses: {
      name: bookingData.customer.name,
      email: bookingData.customer.email,
      phone: bookingData.customer.phone,
      partySize: bookingData.partySize,
      eventType: bookingData.eventType,
      specialRequests: bookingData.specialRequests,
    },
    metadata: {
      bookingSource: 'web',
      restaurantId: bookingData.restaurant.id,
      branchId: bookingData.branch.id,
      paymentId: bookingData.paymentId,
    },
    language: 'es',
    timeZone: bookingData.branch.timeZone,
  })),
});

if (!response.ok) {

```

```
    const error = await response.json();
    throw new Error(error.message || 'Failed to create
booking');
}
```

```
    const result = await response.json();
    return result.id;
}
```

Webhooks de [Cal.com](https://cal.com):

typescript

```
// Escuchar eventos de Cal.com
// POST /api/webhooks/calcom
```

```
export default async function handler(req: NextApiRequest,
res: NextApiResponse) {
    const { event, payload } = req.body;
```

```
    switch (event) {
        case 'BOOKING_CREATED':
            // Enviar email de confirmación
            await sendConfirmationEmail(payload);
            // Enviar WhatsApp de confirmación
            await sendWhatsAppConfirmation(payload);
            // Actualizar estado en base de datos
            await updateBookingStatus(payload.id, 'confirmed');
            break;
```

```
        case 'BOOKING_CANCELLED':
            // Procesar reembolso si aplica
            if (payload.paymentId) {
                await processRefund(payload.paymentId);
            }
            // Enviar notificación de cancelación
            await sendCancellationNotification(payload);
            break;
```

```
        case 'BOOKING_RESCHEDULED':
            // Enviar notificación de reprogramación
            await sendRescheduleNotification(payload);
            break;
    }
```

```
    res.status(200).json({ received: true });
}
```

MANEJO DE ERRORES Y ESTADOS

ESTADOS DE CARGA:

```
typescript
// Skeleton loaders para cada paso
<StepOneSkeleton />
<DateTimeSelectorSkeleton />
<CustomerFormSkeleton />
<PaymentFormSkeleton />
```

MANEJO DE ERRORES:

```
typescript
// Error boundary global
class BookingErrorBoundary extends React.Component {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error: Error) {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      return <ErrorFallback error={this.state.error} />;
    }
    return this.props.children;
  }
}

// Error fallback component
function ErrorFallback({ error }: { error: Error }) {
  return (
    <div className="error-container">
      <h2>Algo salió mal</h2>
      <p>{error.message}</p>
      <button onClick={() => window.location.reload()}>
        Intentar de nuevo
      </button>
    </div>
  );
}
```

VALIDACIONES DE DISPONIBILIDAD EN TIEMPO REAL:

```
typescript
```

```
// Verificar disponibilidad antes de confirmar
async function verifyAvailability(bookingData: BookingState):
Promise<boolean> {
  const response = await fetch(

`/api/restaurants/${bookingData.restaurant.id}/branches/${book
ingData.branch.id}/verify-availability`,
  {
    method: 'POST',
    body: JSON.stringify({
      date: bookingData.selectedDate,
      time: bookingData.selectedTime,
      partySize: bookingData.partySize,
    }),
  }
);

  const result = await response.json();
  return result.available;
}
```

ACCESIBILIDAD (WCAG 2.1 AA)

REQUISITOS:

- Todos los inputs tienen labels asociados
- Navegación completa por teclado (Tab, Enter, Escape)
- Focus visible en todos los elementos interactivos
- Contraste de colores mínimo 4.5:1 (texto normal)
- Textos alternativos en imágenes y iconos
- ARIA labels en botones sin texto
- Anuncios de errores en tiempo real (aria-live)
- Tamaño de fuente ajustable (hasta 200%)
- Soporte para lectores de pantalla (VoiceOver, NVDA, JAWS)

IMPLEMENTACIÓN:

```
typescript
// Ejemplo de input accesible
<label htmlFor="customer-name" className="sr-only">
  Nombre completo
</label>
<input
  id="customer-name"
```

```

    type="text"
    aria-required="true"
    aria-invalid={errors.name ? "true" : "false"}
    aria-describedby={errors.name ? "name-error" : undefined}
    className={cn(
      "input-field",
      errors.name && "border-red-500"
    )}
  />
  {errors.name && (
    <p id="name-error" className="error-message" role="alert">
      {errors.name}
    </p>
  )}
)}

```

RESPONSIVE DESIGN (MOBILE-FIRST)

BREAKPOINTS:

```

css
/* Mobile first */
@media (min-width: 640px) {
  /* Tablet */
}

@media (min-width: 768px) {
  /* Desktop pequeño */
}

@media (min-width: 1024px) {
  /* Desktop */
}

@media (min-width: 1280px) {
  /* Desktop grande */
}

```

ADAPTACIONES POR DISPOSITIVO:

Móvil:

- Una columna
- Botones grandes (min 44x44px)
- Navegación por gestos (swipe para cambiar de paso)

- Teclado numérico para campos de número
- Bottom sheet para modales

Tablet:

- Dos columnas donde aplique
- Grid de time slots más amplio
- Preview lateral del resumen

Desktop:

- Layout de dos columnas (configuración + preview)
- Hover effects
- Tooltips en lugar de texto explicativo inline
- Atajos de teclado (Ctrl+Enter para continuar)

OPTIMIZACIÓN DE RENDIMIENTO

CODE SPLITTING:

```
typescript
// Lazy load de componentes pesados
const DateTimeSelector = dynamic(() =>
import('./DateTimeSelector'), {
  loading: () => <DateTimeSelectorSkeleton />,
});

const PaymentForm = dynamic(() => import('./PaymentForm'), {
  loading: () => <PaymentFormSkeleton />,
});
```

IMAGE OPTIMIZATION:

```
typescript
import Image from 'next/image';
```

```
<Image
  src={branch.imageUrl}
  alt={branch.name}
  width={400}
  height={300}
  loading="lazy"
  placeholder="blur"
```

```
    blurDataURL={branch.photoBlurHash}  
  />
```

API OPTIMIZATION:

```
typescript  
// SWR para caché de disponibilidad  
import useSWR from 'swr';  
  
function useAvailability(branchId: string, date: string) {  
  const { data, error, isLoading } = useSWR(  
    `/api/branches/${branchId}/availability?date=${date}`,  
    fetcher,  
    {  
      revalidateOnFocus: false,  
      dedupingInterval: 60000, // 1 minuto  
    }  
  );  
  
  return { data, error, isLoading };  
}
```

TESTING

TESTS UNITARIOS:

```
// tests/PartySizeSelector.test.tsx  
describe('PartySizeSelector', () => {  
  it('debe mostrar botones del 1 al 8', () => {  
    render(<PartySizeSelector value={1} onChange={() => {}}  
      maxCapacity={50} />);  
    expect(screen.getByText('1')).toBeInTheDocument();  
    expect(screen.getByText('8')).toBeInTheDocument();  
  });  
  
  it('debe llamar onChange al hacer click en un número', () =>  
  {  
    const onChange = jest.fn();  
    render(<PartySizeSelector value={1} onChange={onChange}  
      maxCapacity={50} />);  
    fireEvent.click(screen.getByText('4'));  
    expect(onChange).toHaveBeenCalledWith(4);  
  });  
});
```



```

    it('no debe permitir seleccionar más que la capacidad
máxima', () => {
      render(<PartySizeSelector value={1} onChange={() => {}}
maxCapacity={5} />);
      expect(screen.getByText('6')).not.toBeInTheDocument();
    });
  });
});

```

TESTS DE INTEGRACIÓN:

```

typescript
// tests/BookingFlow.test.tsx
describe('Flujo completo de reserva', () => {
  it('debe completar una reserva exitosamente', async () => {
    render(<BookingPage />);

    // Paso 1: Seleccionar número de personas
    fireEvent.click(screen.getByText('4'));
    fireEvent.click(screen.getByText('Siguiente'));

    // Paso 2: Seleccionar fecha y hora
    fireEvent.click(screen.getByText('15'));
    fireEvent.click(screen.getByText('20:00'));
    fireEvent.click(screen.getByText('Siguiente'));

    // Paso 3: Datos del cliente
    fireEvent.change(screen.getByLabelText('Nombre'), {
target: { value: 'Juan Pérez' } });
    fireEvent.change(screen.getByLabelText('Email'), { target:
{ value: 'juan@email.com' } });
    fireEvent.change(screen.getByLabelText('Teléfono'), {
target: { value: '+573001234567' } });
    fireEvent.click(screen.getByText('Siguiente'));

    // Paso 4: Confirmar
    fireEvent.click(screen.getByText('Confirmar reserva'));

    // Paso 5: Ver confirmación
    await waitFor(() => {
      expect(screen.getByText('¡Reserva
confirmada!')).toBeInTheDocument();
    });
  });
});

```

TESTS E2E (Playwright):

```
typescript
// e2e/booking.spec.ts
import { test, expect } from '@playwright/test';

test('flujo completo de reserva', async ({ page }) => {
  await page.goto('/reservar');

  // Seleccionar sede
  await page.click('[data-testid="branch-card-1"]');

  // Seleccionar número de personas
  await page.click('text=4');
  await page.click('text=Siguiente');

  // Seleccionar fecha y hora
  await page.click('[data-testid="date-15"]');
  await page.click('text=20:00');
  await page.click('text=Siguiente');

  // Completar datos
  await page.fill('[name="name"]', 'Juan Pérez');
  await page.fill('[name="email"]', 'juan@email.com');
  await page.fill('[name="phone"]', '+573001234567');
  await page.click('text=Siguiente');

  // Confirmar
  await page.click('text=Confirmar reserva');

  // Verificar confirmación
  await expect(page.locator('text=;Reserva confirmada!')).toBeVisible();
});
```

RESUMEN DE ENTREGABLES ESPERADOS

Después de implementar este módulo, debes tener:

✓ Frontend:

- Flujo completo de 5 pasos funcional
- Selección de sede con geolocalización
- Calendario con indicadores de disponibilidad
- Grid de time slots con filtros

- Formulario de cliente (guest-first)
- Integración con pasarelas de pago
- Generación de QR de confirmación
- Descarga de QR y agregar a calendario
- Sistema de feedback

✓ Backend:

- API REST completa para reservas
- Integración con Cal.com
- Webhooks para eventos de reservas
- Validación de disponibilidad en tiempo real
- Procesamiento de pagos seguro
- Generación de QRs
- Envío de emails y notificaciones

✓ UX/UI:

- Diseño responsive (mobile-first)
- Animaciones suaves entre pasos
- Loading states y skeleton loaders
- Error handling robusto
- Accesibilidad WCAG 2.1 AA
- Optimización de rendimiento

✓ Testing:

- Tests unitarios de componentes
- Tests de integración de flujos
- Tests E2E con Playwright
- Tests de accesibilidad

INSTRUCCIONES PARA ANTIGRAVITY

1. Comienza con el estado global (Zustand store)
2. Implementa el routing entre pasos (React Router o Next.js App Router)
3. Desarrolla paso por paso (Paso 0 → Paso 5)
4. Integra con Cal.com para disponibilidad y creación de reservas
5. Agrega las pasarelas de pago según configuración del restaurante
6. Implementa la generación de QR y descargas
7. Asegura la calidad (tests, validaciones, error handling)
8. Optimiza el rendimiento (code splitting, lazy loading, caché)

Prioridad:

1. Flujo básico (sin pago)
2. Calendario y time slots
3. Formulario de cliente
4. Integración con Cal.com
5. Pasarelas de pago
6. QR y confirmación
7. Características avanzadas (geolocalización, feedback, etc.)

Nota: Cada paso debe ser funcional independientemente, pero integrarse perfectamente con los demás. El flujo debe ser fluido y sin fricciones.